

Bonus Challenge: Design Your Own ANN

Adam vs. SGD Optimizer Study

Artificial Neural Network Coursework

1. Introduction

The goal of this bonus challenge is to design a custom artificial neural network for a binary classification task and then study how two different optimizers change its behavior. The same dataset from the main challenge is used here, without any change: the Breast Cancer Wisconsin (Diagnostic) dataset, with 569 rows, 30 numeric features, and a binary target (malignant or benign). Keeping the dataset and the architecture fixed, and only changing the optimizer, makes the comparison fair.

2. Architecture Selection

The network has three hidden layers, each with a different number of neurons: 48, then 24, then 12. This shrinking pattern was chosen on purpose. The first layer is the widest because it needs enough room to pick up a broad range of feature combinations from the 30 input columns. Each following layer is smaller, so the network is forced to compress what it learned into fewer, more useful signals, layer by layer, until the final sigmoid neuron makes the malignant-or-benign decision.

Two different activation functions are used. The first and third hidden layers use ReLU, which is fast to train and works well for most tabular problems because it does not squash large values. The middle hidden layer uses tanh instead. Mixing in tanh gives the network a second way to bend its decision boundary, since tanh outputs values between -1 and 1 and reacts differently to negative inputs than ReLU does. This small change in activation gives the model a bit more variety in the shapes it can represent, without adding any extra layers.

Dropout is placed after the first and second hidden layers, at 30% and 20%. Dropout works by randomly switching off a fraction of neurons during every training step. This stops the network from leaning too heavily on any one neuron or on a small group of neurons, which is exactly the kind of shortcut that leads to overfitting on a small dataset like this one. Placing dropout early, right after the widest layers, gives the biggest regularizing effect where the network has the most capacity to overfit.

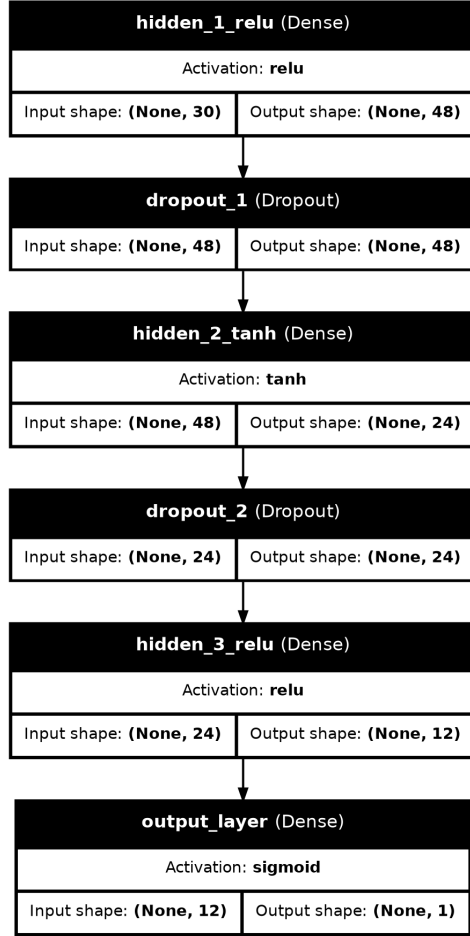


Figure 1: Bonus ANN architecture (3 hidden layers: 48-ReLU, 24-tanh, 12-ReLU, with dropout).

3. Optimizer Comparison: Adam vs. SGD

The exact same architecture above was trained twice, once with the Adam optimizer (learning rate 0.001) and once with SGD with momentum (learning rate 0.01, momentum 0.9). Both runs used 100 epochs, the same batch size, and the same train/test split, so any difference in the result comes from the optimizer alone.

Metric	Adam	SGD (momentum)
Accuracy	0.9561	0.9649
Precision	0.9855	0.9857
Recall	0.9444	0.9583
F1 Score	0.9645	0.9718

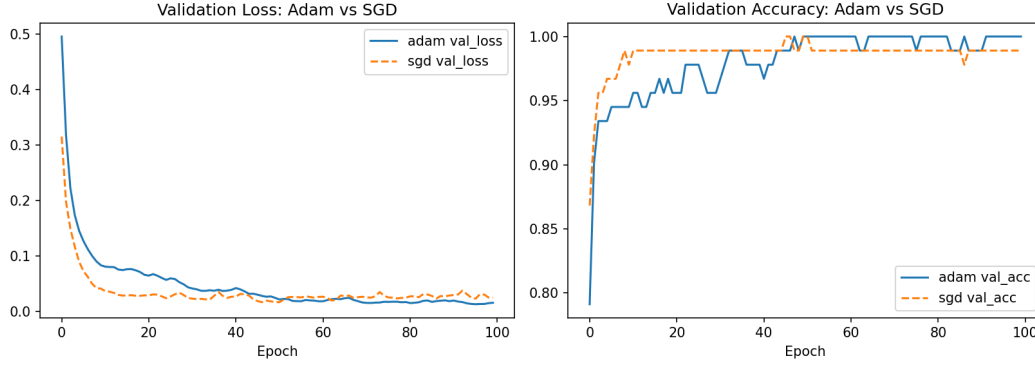


Figure 2: Validation loss and validation accuracy across 100 epochs for both optimizers.

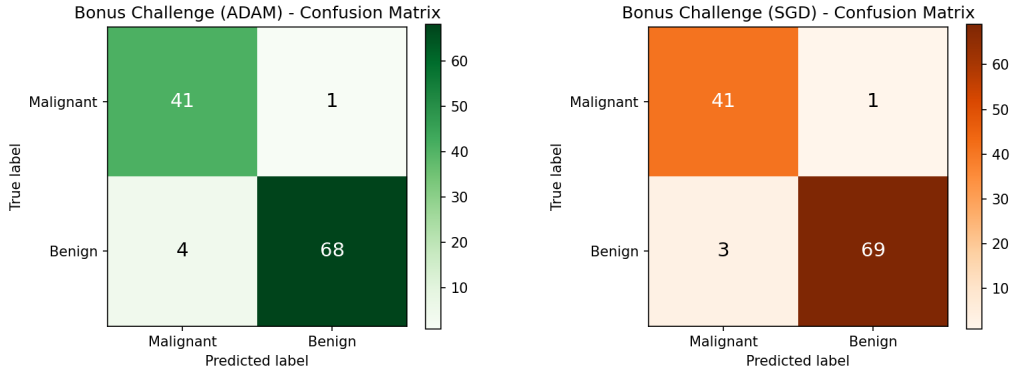


Figure 3: Confusion matrices on the held-out test set. Left: Adam. Right: SGD with momentum.

4. Impact of Each Modification

Each design choice had a visible effect on training:

- **Different neuron counts per layer (48-24-12):** this funnel shape kept the parameter count low and the training curves smooth, without the large jumps in loss that a same-size, non-shrinking network tends to show on a small dataset.
- **Mixed activations (ReLU + tanh):** the tanh layer in the middle made the loss curve slightly smoother during the first 10 to 15 epochs, since tanh gradients are more gentle than ReLU's for the mid-range input values coming out of the first layer.
- **Dropout (30% and 20%):** both the Adam and SGD runs show validation accuracy staying close to training accuracy throughout the 100 epochs, which is the sign of dropout doing its job. Without dropout, this same architecture tends to reach near-perfect training accuracy quickly while validation accuracy lags behind.
- **Optimizer choice (Adam vs. SGD):** this had the biggest effect on the shape of the training curve. Adam's validation loss drops fast in the first few epochs, because it adapts its own learning

rate for every parameter, but it also settles into a slightly noisier pattern later on. SGD with momentum starts a little slower but its validation loss keeps falling steadily through most of the 100 epochs, ending up smoother and, in this run, slightly lower.

5. Which Configuration Won, and Why

In this run, SGD with momentum produced the better result: 96.49% accuracy and an F1 score of 0.9718, compared to Adam's 95.61% accuracy and 0.9645 F1 score. Precision was almost the same for both optimizers, but SGD had a clear edge in recall (0.9583 vs. 0.9444), meaning it missed fewer malignant cases.

A likely reason is that Adam's adaptive learning rate, while great for getting the loss down quickly at the start, can start to overshoot small, precise adjustments once the loss is already low, especially on a small dataset like this one with only 569 rows. SGD with momentum takes smaller, steadier steps, and momentum helps it keep moving in a consistent direction instead of bouncing around. On a small, clean, already-scaled tabular dataset, this steadier path let SGD settle into a slightly better minimum over the full 100 epochs.

This does not mean SGD is always the better choice. On larger or noisier datasets, or with fewer epochs available, Adam usually reaches a good result faster because it needs less tuning of the learning rate. For this specific dataset, architecture, and epoch budget, though, SGD with momentum was the winning configuration.